

Build Prompt: Private Social Messaging Web App

Build a complete, production-ready private social messaging web application designed for a small private community. The UI should feel premium and modern — inspired by Messenger, Telegram, Discord, and Instagram — but with its own original visual identity. It must be fully responsive on Android phones, tablets, and desktop browsers.

1. TECHNOLOGY STACK (LOCKED — do not deviate)

- **Frontend:** React + Vite (do not use vanilla JS)
- **Styling:** modern CSS, fully responsive
- **Backend:** Supabase (PostgreSQL, Auth, Realtime, Storage)
- **Deployment target:** Vercel
- **Secrets:** environment variables only — never hardcode any secret, key, or admin code anywhere in the codebase

- **Do not** use any Replit-specific hosting, database, or deployment integrations — keep the build framework-agnostic so it deploys cleanly to Vercel with `npm install`, `npm run dev`, `npm run build`

Provide a complete `README.md` covering: installation, Supabase setup, database setup, environment variables, storage configuration, RLS policies, local development, Vercel deployment, and security considerations.

2. SECURITY MODEL (read this before building anything else)

This app uses **strong server-side security**, not zero-knowledge end-to-end encryption. Be explicit about this in the README and anywhere the UI references security — do not claim "end-to-end encrypted" or "unhackable."

- Messages are encrypted at rest and in transit (TLS + Supabase encryption-at-rest)
- Every table has Row Level Security (RLS); no sensitive operation may rely on frontend-only checks
- Message content is queryable server-side, which is what enables message search, mention

detection, and admin message review below

- Never expose the Supabase service-role key, database credentials, or any admin/emergency secret in frontend code, GitHub, or API responses

3. ACCOUNT SYSTEM

Users create accounts with: username, display name, email, password, profile picture, optional bio.

Every account automatically receives a unique **6-digit Friend Code** (e.g. 482731):

- Exactly 6 digits, unique, immutable by the user (admin can reset it)
- Never used as authentication — it is a discovery identifier only
- Never expose internal database IDs publicly
- Rate-limit Friend Code lookups server-side to prevent enumeration/abuse

Searching a code shows: display name, @username, profile picture, bio (respecting privacy settings) — then the searcher can send a friend request.

4. FRIEND SYSTEM

Implement: add by code, send/accept/decline/cancel request, remove friend, block/unblock.

States: NONE , PENDING_OUTGOING ,
PENDING_INCOMING , FRIENDS , BLOCKED

- No duplicate requests, no self-requests
 - Notify user on incoming request
-

5. PROFILE

Profile page shows: picture, display name, username, Friend Code (with Copy button), bio, joined date, friend count, posts, online status.

Include an **Add Friend** button and user-controlled privacy settings for what non-friends can see.

6. MESSAGING (1:1)

- Text, emoji, reply, edit own message, delete own message, timestamps
- Delivered/read status, typing indicator, online/offline indicator
- Server-side message search (enabled by the security model above)

- Virtualized/paginated message history, scroll-to-latest, unread counts
 - Pin conversation, mute conversation
 - Realtime updates via Supabase Realtime
-

7. GROUP CHAT

Group name, picture, description, members, admins. Add/remove members, leave group, edit group info, group permissions, mute, message search. Only authorized members can access group messages — enforce via RLS.

8. SOCIAL FEED / POSTS

Posts support text, multiple images, emoji.

Users can: like/unlike, comment, delete own post, edit own post, report a post, copy post link.

Privacy per post: Friends / Only Me / Everyone in the community. Feed prioritizes friends' posts. Enforce visibility via RLS — never trust frontend filtering alone.

9. COMMENTS

Realtime comments with text, timestamp, author, avatar, delete own comment, like comment. Users can only modify their own comments (RLS-enforced).

10. NOTIFICATIONS

Notify on: friend request, request accepted, new message, group invite, mention, post like/comment. Unread badge counter. Mention detection can run server-side since message content is queryable (per the security model).

11. SEARCH

Global search by Friend Code, username, display name, friends, and posts (where permitted). Friend Code is the most reliable way to find a specific user. Debounce search queries.

12. SINGLE ADMIN SYSTEM

There is exactly **one** administrator account for this community. The admin has full moderation and account-management capability. Keep admin logic in clearly separated, server-verified code paths — never

trust a frontend "is admin" flag alone; always re-check role server-side via RLS/policy.

Admin can:

- View community statistics
- Manage reported content, including viewing the content of reported messages/posts
- View message content generally, for safety/moderation oversight of this private community
- Suspend/restore accounts, ban users, remove posts
- Reset a user's Friend Code
- Manage friend relationships when necessary

Log admin content-access actions (what was viewed, when, why if provided) so there's an audit trail even though access itself isn't blocked.

13. EMERGENCY ADMIN ACCESS (break-glass)

A hidden, unadvertised admin-only route:

`/admin/emergency` — not linked anywhere in the normal UI.

Emergency code handling:

- Store only as a Vercel server-side environment variable (`EMERGENCY_ADMIN_CODE`), set manually by the admin after deployment
- Never hardcoded in frontend or backend source, never committed to Git, never sent to the browser, never returned in any API response, never logged in plaintext
- Verify server-side only; store a hash if persistence is needed at all
- Rate-limit attempts with temporary lockout after repeated failures

Break-glass flow requires:

1. Authenticated admin account
2. Emergency code entry
3. Reason for emergency access (free text)
4. Confirmation step

Emergency actions available:

- View account info, revoke sessions, force logout all devices
- Reset account password/email through secure recovery
- Temporarily disable an account
- Review recent login activity and active sessions, remove suspicious devices

- Temporary impersonation — while active, show a persistent banner: "⚠ EMERGENCY ADMIN SESSION — you are viewing this account as administrator" with a clear **Exit Emergency Session** control

Audit log (tamper-resistant, admin cannot delete entries):

Record admin ID, target user ID, timestamp, action, reason, IP address (capture via a Vercel serverless/edge function, not client-side), session ID, and success/failure — for every emergency action.

14. DATABASE

SQL migrations for: `profiles`, `friendships`, `friend_requests`, `conversations`, `conversation_members`, `messages`, `posts`, `comments`, `likes`, `notifications`, `blocks`, `reports`, `user_settings`, `admin_audit_log`.

- UUIDs for all internal IDs (never the Friend Code as a primary key)
- Proper primary/foreign keys, unique constraints, indexes, timestamps
- RLS policies on every table, including explicit admin-access policies where the single admin

role needs broader read access (e.g. reports, messages for moderation)

15. MEDIA

Supabase Storage for profile pictures and post images. Validate file type/size, enforce upload permissions, generate safe filenames, apply storage policies so private media isn't publicly accessible unless intended.

16. PRIVACY DETERRENTS (UI-level, honestly labeled)

Where technically feasible: discourage screenshots, limit easy image dragging/download, disable context menu on sensitive content, optional privacy mode, blur content on window blur. The UI must describe these as **deterrents**, not guarantees — browsers cannot reliably block screenshots, and the app must not claim otherwise.

17. RESPONSIVE UI

- Smooth animations, rounded cards, clean typography, modern icons, good spacing
 - Beautiful empty states, skeleton loading, toasts, modals
 - Desktop: sidebar nav + main content + details panel
 - Mobile: bottom nav bar (Home | Messages | Friends | Notifications | Profile)
 - Dark mode, light mode, system preference — remembered per user, thoughtfully designed (not just inverted colors)
-

18. CHAT UI

Clear incoming/outgoing bubble distinction, avatars, timestamps, read indicators, reply previews, reactions, message menu. Long messages wrap correctly. Images open in a proper media viewer.

19. ERROR HANDLING & PERFORMANCE

Gracefully handle: lost connection, Supabase errors, failed uploads, expired sessions, invalid codes, duplicate requests, deleted users, unauthorized

access — with human-friendly messages, never raw DB errors.

Performance: pagination, lazy loading, image compression, optimized/indexed queries, realtime listener cleanup, debounced search. Never load full message/post history at once.

20. VERCEL DEPLOYMENT

Include `.env.example` :

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_PUBLISHABLE_KEY=
```

Plus a `.gitignore` that excludes `.env`. Never commit `.env`. Must build and deploy to Vercel without source changes: `npm install`, `npm run dev`, `npm run build`.

21. FINAL SECURITY REVIEW (before calling it done)

Check for: exposed secrets, insecure queries, missing RLS, unauthorized message/profile access, ID enumeration, XSS, unsafe HTML rendering, malicious

upload handling, insecure storage policies, missing auth checks, client-side-only authorization.

Do not describe the app as "military-grade," "unhackable," or "100% secure." Include a clear, honest security disclaimer in the README.

22. BUILD ORDER

1. Project structure & Supabase config
2. Authentication
3. Profile creation + Friend Code
4. Friend requests/list
5. Private messaging + Realtime
6. Groups
7. Posts + Comments
8. Notifications
9. Media uploads
10. Security review pass (RLS, storage policies)
11. Single admin panel
12. Emergency admin access
13. Responsive/dark-mode polish
14. Testing
15. Vercel deployment config

Verify the app still builds after each stage. Do not leave placeholder/non-functional buttons — every major UI element should be wired up.

At the end, provide: complete file structure, full SQL schema with RLS policies, environment variable list, deployment steps, security checklist, and a testing checklist.